



Container Orchestration Storage

Storage Class

 Copy page

This article explores storage classes in Kubernetes, focusing on static and dynamic provisioning for effective storage resource management.

In this article, we explore how storage classes work in Kubernetes. We'll build on the basic concepts of creating PersistentVolumes (PVs), PersistentVolumeClaims (PVCs), and using PVCs within pod definitions. Understanding static and dynamic provisioning methods is essential for effective storage resource management in your Kubernetes environment.

Kubernetes storage classes simplify storage management by automating the provisioning process. Whether you use static provisioning or dynamic provisioning, the concepts remain similar, with dynamic provisioning offering automated PV creation.

Static Provisioning

Static provisioning requires manual setup. First, you create the persistent disk on your cloud provider (for example, on Google Cloud), then you manually create the PV definition using the exact same disk name. Each application that requires storage needs you to pre-provision the disk and create the corresponding PV configuration.

Below are the YAML definitions for the PV, PVC, and Pod using static provisioning:



>

```
name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

```
# pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```





>

```
name: random-number-generator
spec:
  containers:
    - image: alpine
      name: alpine
      command: ["/bin/sh", "-c"]
      args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
      volumeMounts:
        - mountPath: /opt
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: myclaim
```

Before applying the PV definition, ensure that the underlying disk exists. For example, on Google Cloud you would run:

```
gcloud beta compute disks create \
  --size 1GB \
  --region us-east1 \
  pd-disk
```



Dynamic Provisioning with Storage Classes

Dynamic provisioning automates the storage creation process. Instead of manually creating PVs, you use a storage class that defines a provisioner (like Google Cloud's persistent disk provisioner) to automatically create and attach a disk when a claim is made.

The dynamic provisioning workflow is as follows:



>

field.

3. When a PVC is created, the storage class's provisioner dynamically creates a new disk with the defined specifications, automatically generates a corresponding PV, and binds the PVC to that PV.

Below is an example for dynamic provisioning using a storage class:

```
# sc-definition.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
```



```
# pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: google-storage
resources:
  requests:
    storage: 500Mi
```





>

```
name: random-number-generator
spec:
  containers:
    - image: alpine
      name: alpine
      command: ["/bin/sh", "-c"]
      args: ["shuf -i 0-100 -n 1 >> /opt/data-volume/output.txt"]
      volumeMounts:
        - mountPath: /opt
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: myclaim
```

With dynamic provisioning, there is no need for pre-created PV definitions; the storage class takes care of handling PV creation automatically when the PVC is submitted.

Multiple Storage Classes

A key advantage of using storage classes is the ability to define different service levels tailored to various performance and replication needs. For example, you might use:

Storage Class	Description
Silver	Uses standard persistent disks
Gold	Uses SSD persistent disks
Platinum	Uses SSD persistent disks with regional replication



>

```
metadata:
  name: silver
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  replication-type: none
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gold
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
  replication-type: none
```



```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: platinum
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
  replication-type: regional-pd
```



When creating a PVC, simply specify the desired storage class name and Kubernetes will dynamically provision the volume with the characteristics defined.



>



Watch Video

[Learn more >](#)

Was this page helpful?

Yes

No

Persistent Volume Claims

[< Previous](#)

Autoscaling

[Next >](#)

Powered by **mintlify**